

CHAPTER - 4

PROGRAMMING CONCEPT USING C++

OVERVIEW OF PROGRAMMING CONCEPTS

4.1 Introduction to C++ Programming Language

C++ is a powerful, high-performance programming language that was developed by Bjarne Stroustrup in 1979 at Bell Labs. Initially designed as an extension of the C programming language, C++ was intended to provide additional features for system-level programming and to support object-oriented programming (OOP) concepts. Over the years, C++ has evolved significantly and is now considered one of the most versatile and widely used programming languages in the world.



Key Features of C++

1. Object-Oriented Programming (OOP): C++ was among the first languages to introduce the concept of OOP, which emphasizes organizing software design around data, or objects, rather than functions and logic. The core principles of OOP in C++ include:

- **Encapsulation:** Bundling data and methods that operate on the data within one unit, called a class.
- **Inheritance:** The ability to create new classes that are based on existing classes, enabling code reusability and the creation of a hierarchy.
- **Polymorphism:** The ability to process objects differently based on their data type or class, often through method overriding and interfaces.
- **Abstraction:** Simplifying complex systems by modeling classes appropriate to the problem.

2. Low-Level Manipulation: Despite being a high-level language, C++ allows for fine-grained control over system resources through features like pointers, direct memory access, and manual memory management. This makes it ideal for system programming, including writing operating systems, device drivers, and other performance-critical applications.

3. Rich Standard Library: C++ includes a vast standard library known as the Standard Template Library (STL), which provides many built-in functions and data structures like vectors, lists, queues, and stacks. The STL supports generic programming through templates, which allows for the creation of functions and classes that work with any data type.

4. Portability: C++ code can be compiled and run on many different platforms without modification, making it highly portable. This is particularly valuable for developing cross-platform applications.

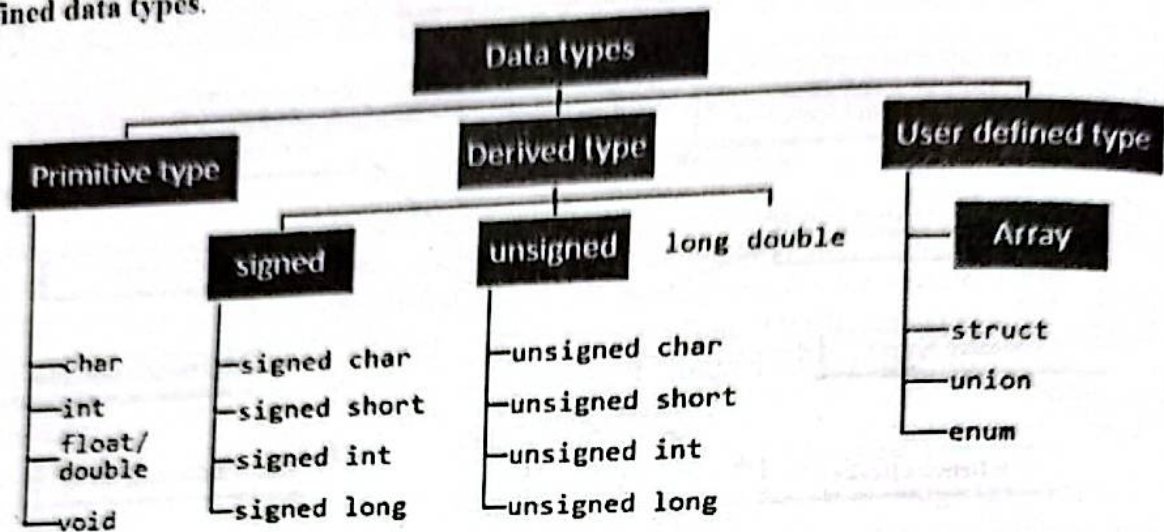
5. Performance: One of C++'s most notable features is its efficiency and performance. C++ code is typically compiled into machine code, which runs directly on the hardware, leading to faster execution times. This makes C++ a preferred choice for high-performance applications such as games, real-time simulations, and embedded systems.

6. Multi-Paradigm Language: C++ supports multiple programming paradigms, including procedural programming, object-oriented programming, and generic programming. This flexibility allows developers to choose the most appropriate style for their specific needs.

7. Exception Handling: C++ includes robust error-handling mechanisms through the use of exceptions. This allows developers to write more reliable and maintainable code by providing a way to handle runtime errors gracefully.

4.2 Data Types in C++

C++ provides a variety of data types that allow programmers to store and manipulate different kinds of data. These data types can be categorized into several groups: **basic data types**, **derived data types**, and **user-defined data types**.



1. Basic Data Types:

• Integer Types:

- ✓ **int**: The most commonly used integer type, typically used for storing whole numbers. The size of int is usually 4 bytes, which allows it to store values ranging from approximately -2 billion to 2 billion.
- ✓ **short**: A smaller integer type, usually 2 bytes in size. It can store values ranging from -32,768 to 32,767.
- ✓ **long**: Used for larger integers. Its size is at least as large as int, typically 4 or 8 bytes, depending on the system.
- ✓ **long long**: An extended integer type that typically uses 8 bytes of memory, capable of storing very large integers.

• Floating-Point Types:

- ✓ **float**: Used for storing single-precision floating-point numbers. It usually occupies 4 bytes and is precise up to 7 decimal digits.
- ✓ **double**: Used for storing double-precision floating-point numbers. It typically occupies 8 bytes and is precise up to 15 decimal digits.
- ✓ **long double**: Provides even greater precision and range than double. The size can vary depending on the compiler, but it's typically 8 or 16 bytes.

• Character Type:

- ✓ **char**: Used for storing single characters. It usually occupies 1 byte and can store values from -128 to 127 or 0 to 255, depending on whether it's signed or unsigned.

• Boolean Type:

- ✓ **bool**: Introduced in C++, bool is used for storing Boolean values, true or false. It typically occupies 1 byte.

• Void Type:

- ✓ **void**: Represents the absence of type. It's used for functions that do not return a value and for pointers to indicate that they point to a type that is not yet specified.

- *. Dereferences a pointer to access the value at the pointed-to address. Example: `*ptr`.
- `sizeof`: A unary operator that returns the size of a variable, type or object. Example: `sizeof(int)`.

Key Points:

- C++ offers a wide range of data types and operators, providing flexibility and control over memory and data manipulation.
- Understanding the different data types is crucial for efficient memory usage, especially in performance-critical applications.
- Operators in C++ allow for powerful expressions and are fundamental to writing effective and concise code.
- User-defined data types like classes and structs form the basis of Object-Oriented Programming in C++, allowing developers to create complex and reusable code components.

4.3 Input and Output Streams in C++

In C++, input and output (I/O) operations are handled using streams. A stream is an abstraction that represents a sequence of bytes, which can be used for reading (input) or writing (output). The C++ Standard Library provides several classes and functions to manage I/O operations, which are essential for interacting with users, files, and other devices.

Basic I/O Streams:

C++ includes several predefined I/O stream objects, which are part of the `<iostream>` library:

- `std::cin`: Standard input stream, typically used to read input from the keyboard.
- `std::cout`: Standard output stream, typically used to print output to the console.
- `std::cerr`: Standard error stream, used to output error messages. Unlike `std::cout`, it is not buffered, meaning it displays errors immediately.
- `std::clog`: Standard log stream, used for logging messages. It is buffered, so the output may not appear immediately.


```

std::cout << "Integer: " << intValue << std::endl;
std::cout << "Float: " << floatValue << std::endl;
std::cout << "String: " << strValue << std::endl;
return 0;
}

```

In this example, `std::stringstream` is used to read values from a string as if they were being input from the console. The values are extracted using the `>>` operator.

Conclusion:

Streams in C++ provide a flexible and powerful way to handle input and output operations. Whether you're working with the console, files, or strings, C++ offers a consistent and intuitive interface for performing I/O tasks. Understanding how to use these streams is essential for developing interactive applications, handling files, and processing data efficiently.

4. Use cascading technique in input output streams

1. Arithmetic Operators:

This program uses basic arithmetic operators (+, -, *, /, %) to perform operations on two integers.

2. Relational Operators:

This program demonstrates the use of relational operators (==, !=, <, >, <=, >=) to compare two integers.

```

std::cout << "a == b: " << (a == b) << std::endl;
std::cout << "a != b: " << (a != b) << std::endl;
std::cout << "a < b: " << (a < b) << std::endl;
std::cout << "a > b: " << (a > b) << std::endl;
std::cout << "a <= b: " << (a <= b) << std::endl;
std::cout << "a >= b: " << (a >= b) << std::endl;

```

3. Logical Operators:

This program uses logical operators (&&, ||, !) to evaluate logical expressions.

```

std::cout << "a && b: " << (a && b) << std::endl;
std::cout << "a || b: " << (a || b) << std::endl;
std::cout << "!a: " << (!a) << std::endl;

```

4. Bitwise Operators:

This program demonstrates the use of bitwise operators (&, |, ^, ~, <<, >>) on integer values.

```

std::cout << "a & b: " << (a & b) << std::endl; // Bitwise AND
std::cout << "a | b: " << (a | b) << std::endl; // Bitwise OR
std::cout << "a ^ b: " << (a ^ b) << std::endl; // Bitwise XOR
std::cout << "~a: " << (~a) << std::endl; // Bitwise NOT
std::cout << "a << 1: " << (a << 1) << std::endl; // Left shift
std::cout << "a >> 1: " << (a >> 1) << std::endl; // Right shift

```

5. Assignment Operators

This program uses assignment operators (=, +=, -=, *=, /=, %=) to modify the value of a variable.

```

#include <iostream>
int main() {
    int a = 10;
    std::cout << "Initial value of a: " << a << std::endl;
    a += 5;
    std::cout << "After a += 5: " << a << std::endl;
}

```


Chapter – 4: Programming Concept Using C++

```
a %= 3; // a = a % 3
```

```
std::cout << "After a %= 3: a = " << a << std::endl;
```

9. Relational and Logical Operators

This program demonstrates the combination of relational and logical operators to evaluate conditions.

```
#include <iostream>
int main() {
    int x = 10, y = 20, z = 30;
    bool result;
    result = (x < y) && (y < z);
    std::cout << "(x < y) && (y < z): " << result << std::endl;
    result = (x > y) || (y < z);
    std::cout << "(x > y) || (y < z): " << result << std::endl;
    result = !(x == y);
    std::cout << "!(x == y): " << result << std::endl;
    return 0;
```

10. Bitwise Operators for Masking:

This program uses bitwise operators for masking and checking specific bits.

```
#include <iostream>
int main() {
    unsigned char flags = 0b00001101; // 13 in decimal, 00001101 in binary
    // Mask to check if the 3rd bit is set
    unsigned char mask = 0b00000100;
    if (flags & mask) {
        std::cout << "The 3rd bit is set." << std::endl;
    } else {
        std::cout << "The 3rd bit is not set." << std::endl;
    }
    // Set the 2nd bit
    flags |= 0b00000010;
    std::cout << "After setting the 2nd bit: " << std::bitset<8>(flags) << std::endl;
    // Clear the 4th bit
    flags &= ~0b00001000;
    std::cout << "After clearing the 4th bit: " << std::bitset<8>(flags) << std::endl;
    return 0;
```

These examples cover a broad range of operators in C++ and demonstrate their use in various contexts. By experimenting with these examples, you can gain a deeper understanding of how operators work in C++.

REVIEW OF CONTROL STRUCTURE

4.5 Selection / Decision Making Structure to Develop Programs

Selection or decision-making structures in C++ allow a program to choose between different courses of action based on the evaluation of a condition. The primary decision-making structures in C++ are:

1. if statement

2. if...else statement

3. else if ladder

6. if Statement

the code block inside the if statement is